YUKTI

Recommended Technology Stack

Project-focused technology decisions for a multi-role platform with citizen reporting, verification, university/industry collaboration, AI-assisted analysis, geographic data, dashboards and deployment.

Decision principle
The stack is selected for requirement fit, beginner-team feasibility, clean integration, reasonable cost, SIH prototype reliability, and room to scale. The goal is not to use every popular technology; the goal is to use the smallest coherent stack that can deliver the required product.

1. Recommended Stack at a Glance

Layer	Selected Technology	Purpose in YUKTI
UI/UX Design	Figma	Create wireframes, user flows, shared design system and high-value screens before/alongside implementation.
Frontend	React.js + Tailwind CSS	Build reusable components, role-based dashboards, forms, tables and dynamic application screens.
Frontend Build	Vite	Fast local development server and production build pipeline for the React app.
Backend	FastAPI + Python	Build REST APIs, application logic, validation, authentication integration and connections to AI/database services.
Database	PostgreSQL	Primary relational database for users, problems, claims, verification, organizations, pipeline and other structured records.
Geographic Data	PostGIS	Store and query coordinates/geometries and support location-based operations on PostgreSQL.
Authentication	JWT + Google OAuth	Use application tokens/sessions for API access and optionally Google sign-in for convenient identity verification.
AI/ML	Python + scikit-learn + Sentence Transformers	Use classical ML for suitable prediction/classification tasks and embeddings for semantic similarity/duplicate detection.
LLM	Gemini API	Natural-language summarization, explanation, recommendation drafting and assistant-style features where generative AI adds value.
Maps	Leaflet + OpenStreetMap-derived map data	Interactive maps, markers, polygons and location-focused views.
Charts	Apache ECharts	Interactive analytics and dashboard visualizations with room for richer data exploration.
File Storage	Supabase Storage	Store images, videos, audio and documents outside the relational database while keeping metadata in PostgreSQL.

Layer	Selected Technology	Purpose in YUKTI
Notifications	Firebase Cloud Messaging	Push notifications for important workflow events; implement after core workflow is stable.
Version Control	Git + GitHub	Branches, pull requests, collaboration, history and code review.
API Testing	Postman	Independently test backend endpoints before frontend integration is complete.
Deployment	Vercel + Render + Supabase	Vercel for frontend, Render for FastAPI, Supabase for managed PostgreSQL/PostGIS/Storage.
API Documentation	OpenAPI + FastAPI Swagger UI	Automatically document and interactively test the API contract.
Monitoring	Sentry	Capture production exceptions and diagnose failures after the core system is working.
Team Management	GitHub Projects	Track backlog, development, review, testing and completed work.

2. Core Web Application

The main application path is intentionally straightforward: React/Vite/Tailwind on the client, FastAPI/Python as the API layer, and PostgreSQL/PostGIS as the data layer. This keeps the architecture understandable to a beginner team while still supporting multiple user roles and dynamic workflows.

3. AI Layer

AI should be split by job rather than used everywhere. Sentence Transformers can turn problem text into embeddings for semantic similarity and duplicate detection. scikit-learn can be used when a trained classification/scoring model is justified. Gemini should handle natural-language tasks such as summaries, explanations and recommendation drafting. Deterministic decisions such as authorization, database status and workflow rules should remain in backend code.

4. Data + Geography

PostgreSQL is the system of record. PostGIS is used only because location-aware operations can benefit from spatial types and queries. Supabase can host the managed PostgreSQL database, PostGIS extension and Storage together, reducing infrastructure fragmentation.

5. Authentication

Use JWT/application sessions for access to your own APIs. Add Google OAuth as an optional sign-in method. Authentication answers *who is the user?*; your own role/authorization rules answer *what can this user do?*.

6. Maps and Visualization

Leaflet handles the interactive map UI. OpenStreetMap-derived data can provide the base map. Apache ECharts handles analytical dashboards such as category counts, status distributions, trends and other metrics. Do not use both ECharts and Recharts unless there is a specific reason.

7. Files and Notifications

Use Supabase Storage for uploaded media and documents. Keep file metadata in PostgreSQL, not large binary files in normal relational tables. Firebase Cloud Messaging is a later-stage enhancement for workflow notifications; it is not a blocker for the first working prototype.

8. Deployment and Operations

A practical deployment split is Vercel (frontend), Render (FastAPI) and Supabase (database, PostGIS and storage). Add Sentry after the application is stable enough to generate meaningful runtime errors. Keep deployment simple until core features are validated.

9. Why this stack is coherent

Frontend: React + Tailwind + Vite
API: FastAPI + Python
Data: PostgreSQL + PostGIS
Files: Supabase Storage
Identity: JWT + optional Google OAuth
AI: scikit-learn + Sentence Transformers + Gemini
Maps: Leaflet + OSM-derived tiles/data
Charts: Apache ECharts
Delivery: Vercel + Render + Supabase

10. What is intentionally NOT added

No MongoDB: the application is relationship-heavy and fits PostgreSQL well.
No D3.js: its low-level flexibility is unnecessary for the current dashboard needs.
No Cloudinary: advanced media transformation is not central to the current requirement; Supabase Storage is simpler for general files.
No second LLM: keep one primary LLM in the first version to reduce cost and integration work.
No Docker requirement at the start: add containerization once deployment consistency becomes a real need.
No early FCM/Sentry work: these are valuable after the core workflow is stable.

11. System Architecture

High-level flow

```
React/Vite/Tailwind → FastAPI → PostgreSQL/PostGIS
                              ■ AI services
                              ■ Supabase Storage
                              ■ Notifications
```

Deployment flow

```
Browser → Vercel (frontend)
Browser → Render (FastAPI API)
FastAPI → Supabase PostgreSQL/PostGIS
FastAPI → Supabase Storage
FastAPI → Gemini / ML services
```

12. Implementation Order

Phase 1: React/Vite/Tailwind cleanup and common component structure.
Phase 2: FastAPI project, PostgreSQL connection, models and schemas.
Phase 3: Authentication, users and role-based authorization.
Phase 4: Problem submission, media upload and core workflow APIs.
Phase 5: Verification, university/industry collaboration and pipeline.
Phase 6: AI/ML services and semantic duplicate detection.
Phase 7: Leaflet maps, ECharts dashboards and dynamic analytics.
Phase 8: Supabase Storage, notifications, reports and polishing.
Phase 9: Deployment, monitoring, testing and final SIH demo hardening.

13. Technology Selection Rule for the Team

Before adding any new tool, ask: **Is it required by the product? Does it reduce meaningful development effort? Can the team learn it in time? Does it integrate cleanly with the existing stack? What are the cost and operational implications?** If the answer is weak, do not add the technology merely because it is popular.

14. Final Stack

Figma · React.js · Tailwind CSS · Vite · FastAPI · Python · PostgreSQL · PostGIS · JWT · Google OAuth · scikit-learn · Sentence Transformers · Gemini · Leaflet · OpenStreetMap-derived map data · Apache ECharts · Supabase Storage · Firebase Cloud Messaging · Git · GitHub · Postman · Vercel · Render · Supabase · OpenAPI/Swagger · Sentry · GitHub Projects